# SharingApp — ATAM Architecture Evaluation

Prepared by: Fariza Arstanbek | 29 October 2026 | Version 1.0

Deliverable: evaluation of all seven supplied Quality Attribute Scenarios, five justified changes to the initial utility tree, and an updated utility-tree diagram. This is a scenario-based architectural analysis, not a completed stakeholder ATAM workshop or a claim of measured runtime quality.

## 1. Evaluation scope and architectural baseline

The delivered Android implementation uses local Gson/file persistence. The target architecture from the previous assignments has Android and browser clients sending HTTP requests to a Web Server, which forwards operations to an Application Server and a shared remote Database; all three server components share one server device and execution environment. These remote services are conceptual, not implemented evidence in the supplied local-storage archive.

The business goals are trustworthy sharing and bidding, consistent access from both clients, responsive workflows and continued maintainability. The main stakeholders considered are owners, borrowers/bidders, operations staff and developers; priorities are provisional analyst judgments because no stakeholder voting results were supplied.

## 2. ATAM method and interpretation

Following the relevant ATAM activities, identify business drivers and architectural approaches, refine the supplied step-5 utility tree with scenarios, analyze scenario/decision relationships, and record risks, non-risks, sensitivity points and tradeoffs. A sensitivity point is a decision/parameter strongly affecting a quality response; a tradeoff affects multiple qualities in competing directions. A risk is an unresolved architectural threat, while a non-risk is a supported decision for a specifically bounded scenario, not proof that the entire system is safe.

Each evaluation below assigns one primary category and provides no more than three explanatory sentences. Q1-Q7 correspond exactly to the scenario order in Quality Attribute Scenarios.pdf; secondary concerns may coexist with the primary classification.

## Evidence and execution limits

Static inspection covered the newly supplied SharingApp_c3_local_storage archive and the supplied user stories. Android SDK/emulator and an implemented remote backend were unavailable, so the app was not built/run and no latency, load, security or concurrency tests are claimed. Numerical targets later in this report are proposed acceptance criteria, clearly separate from observations.

# 3. Scenario evaluation — Q1 to Q4

## Q1 — Search under heavy load | Sensitivity point

Source: End user. Stimulus: Clicks Search. Artifact: Mobile app. Environment: Heavy load.

Required response: Display other users' eligible items loaded from the remote database. Supplied response measure: Time to load remote items.

SP1: Search latency is sensitive to dataset size, query selectivity, payload size and concurrent request load; SearchActivity.keywordSearch() currently reloads local data and scans item words. In the remote design, pagination, indexed server-side filtering and payload limits therefore strongly affect the original sub-second goal. Heavy-load compliance is unverified because no remote implementation or load measurements were supplied.

Trace: E1; updated tree P1.

## Q2 — Create an account and store it externally | Risk

Source: End user. Stimulus: Creates an account with username and email. Artifact: Mobile or web app. Environment: Normal operation.

Required response: Create account and open home page. Supplied response measure: Time until account is stored externally.

RK1: UserList.saveUsers() writes a local JSON file, so the delivered implementation does not establish a shared remote account visible to both clients. The proposed server/database path needs an atomic unique-username constraint, durable commit and confirmation only after success. Otherwise two clients can create conflicting identities or see a success page before the shared account exists.

Trace: E2; updated tree P2, I1.

## Q3 — Log in and load an account externally | Risk

Source: End user. Stimulus: Logs in with username. Artifact: Mobile or web app. Environment: Normal operation.

Required response: Open the home page using the account. Supplied response measure: Time until account is loaded externally.

RK2: LoginActivity.login() obtains an existing user ID from the entered username without proof that the caller owns that identity, while accounts are loaded from local files. A remote version must load the authoritative account and establish a verified session before exposing its home page or allowing owner actions. Username-only identification and fast loading must not be mistaken for secure authentication.

Trace: E2, E3; updated tree P2, S1, I1.

## Q4 — Reject an item with an empty field | Non-risk (bounded)

Source: End user. Stimulus: Attempts item creation with a required field empty. Artifact: Mobile app. Environment: Normal operation.

Required response: Display an input error and do not create the item. Supplied response measure: Time to validate input and show feedback.

NR1: AddItemActivity.saveItem() invokes validateInput(), which sets a field error and returns before creating or saving an item when a required field is the empty string. This local, early rejection supports the exact empty-field scenario without waiting for remote storage. The timing target remains unmeasured, and this finding does not cover whitespace-only strings, malformed numbers or server-side validation.

Trace: E4; updated tree U1.

# 3. Scenario evaluation — Q5 to Q7

## Q5 — Save a valid item from the web application | Tradeoff

Source: End user. Stimulus: Adds a valid item. Artifact: Web app. Environment: Normal operation.

Required response: Display a save confirmation. Supplied response measure: Time for server save and confirmation.

TP1: Waiting for a durable database commit before confirming a web save protects integrity and shared-client visibility but adds network and storage latency to the user response. Confirming optimistically can reduce perceived latency while risking a false success if persistence fails; the local save path already conditions navigation on a successful save result. The target design should favor truthful post-commit confirmation and meet the latency target through efficient queries rather than premature acknowledgement.

Trace: E4, E5; proposed server path; updated tree P3, I1.

## Q6 — Push changes from external systems | Risk

Source: External systems. Stimulus: Changes data used by the applications. Artifact: Web server. Environment: Normal operation.

Required response: Push updated changes to applications. Supplied response measure: Time to send committed changes to clients.

RK3: Observable.notifyObservers() calls in-process observers and cannot push external database changes to separate phones or browsers. Neither the local code nor the earlier conceptual server diagrams define a cross-client event channel, ordering or reconnect recovery. A commit-triggered event path with versioned messages and resynchronization is needed to avoid indefinitely stale application views.

Trace: E6; proposed server path; updated tree I2.

## Q7 — Two users modify the same data | Risk

Source: Two end users. Stimulus: Submit changes to the same record. Artifact: Mobile or web app. Environment: Multiple requests being processed.

Required response: Finish the first request, push its update, and process the next only if consistent. Supplied response measure: Time to send updates while preserving consistency.

RK4: Local list/file replacement and a common server host do not establish serialization or atomic conflict detection across clients, so concurrent bids or item edits can overwrite each other. The application/database boundary needs per-item transactions or version checks that commit the first request, publish its version and reject or revalidate stale later requests. This preserves the supplied ordering requirement; locking and notification coordination also create a secondary throughput/latency tradeoff.

Trace: E5, E6, E7; proposed server path; updated tree R1, I2.

## Risk themes and architectural consequences

Identity and trust: username selection is not proof of ownership, and client-side owner filters are not server authorization. Shared state: local persistence, local observers and multiple-step item/bid updates do not provide cross-device durability, notification or atomicity. Responsiveness: query shape, whole-list transfer, commit time and synchronization strategy must be assessed together; co-location does not eliminate contention or the single-device failure domain.

The common-host availability concern is an additional observation, not an eighth supplied scenario: shared process/device failure can affect all server roles. No availability guarantee is inferred merely from the deployment diagram.

# 4. Five changes to the initial utility tree

| Change | Updated branch / rationale |
|---|---|
| C1 — Refine Performance / Latency | Replace the vague “any task <1 second” leaf with separate search-under-load, external account create/load, and committed item-save leaves P1-P3. Q1-Q3/Q5 have different workload boundaries and must not be validated by one unqualified timing claim. |
| C2 — Strengthen Security / Authentication | Retain the account requirement but require proof of account ownership and a verified server session in S1. Q3 and username-only code expose impersonation risk; this is a proposed architectural improvement beyond identifying a user by username. |
| C3 — Refine Interoperability / Shared Storage | Retain the common shared-data requirement in I1, adding durable remote writes and cross-client visibility, and add bounded change propagation I2. Preserve Mobile App and Web App accessibility I3/I4, because Q2/Q5/Q6 require more than merely drawing a remote database. |
| C4 — Add Usability / Input Feedback | Add U1 for prompt field-specific rejection of empty required fields without saving an item. Q4 supports this behavior, and its medium business priority does not displace authentication or consistency. |
| C5 — Add Reliability / Concurrent-update Integrity | Add R1 for per-record ordered commits, stale-write revalidation/rejection and no lost updates. Q7 is architecturally significant because two clients can change the same item; neither shared storage nor local Observer alone addresses this. |

## Original requirements retained

All original branches remain represented: Performance/Latency (P1-P3), Security/Authentication (S1), Security/Authorization for editing owned items (S2) and preventing bids on owned items (S3), Interoperability/Shared Storage (I1/I2), and Interoperability/Accessibility through mobile and web apps (I3/I4). Existing high/medium business priorities are retained in the first element of each priority pair; newly estimated difficulty is added as the second element.

## Priority notation

Pairs are (business importance, architectural difficulty): H = high, M = medium, L = low. Importance follows the initial H/M labels where a requirement existed; difficulty is an analyst estimate, not stakeholder-approved effort. New synchronization and concurrency leaves are H because stale or lost bids undermine the core sharing workflow; U1 is M because it improves a bounded interaction without replacing correctness controls.

## How to read the updated diagram

The final landscape page is the updated utility tree: Utility → Quality Attribute → Attribute Refinement → ASR/scenario leaf. Leaf IDs connect the diagram to the evaluations and measurable targets. The tree expresses desired requirements, not capabilities already demonstrated by the current local-storage code.

# 5. Proposed response measures and verification

The scenario PDF names response measures but generally supplies no numeric thresholds; the initial tree supplies a general sub-second goal. The following proposed limits make that goal testable and add explicit freshness/integrity measures; all require stakeholder agreement before becoming contractual requirements.

| Scenario / leaf | Proposed measurable criterion and test |
|---|---|
| Q1 / P1 | For 100 concurrent users and 10,000 items under a documented network profile, p95 Search click → first page displayed <1 s. Run a mixed search workload for 10 minutes, checking that results exclude borrowed items and the current user's own items; report latency distribution and error rate. |
| Q2 / P2, I1 | Under a normal workload of 10 concurrent users, p95 create submission → durable account commit and home page <1 s. Verify exactly one account for a username during duplicate submissions, and confirm retrieval from the other client after acknowledgement. |
| Q3 / P2, S1 | Under the same normal workload, p95 valid login submission → authoritative account loaded and home page <1 s. Unauthorized identity attempts must never open another account's session; profile read/write authorization is tested separately. |
| Q4 / U1 | For each required field set to the empty string, display its error within 100 ms of submit and create zero items. Inspect invalid-input paths and execute device UI tests; this does not claim malformed-number/whitespace handling is already correct. |
| Q5 / P3, I1 | For a valid item with no photo and metadata up to 10 KB, p95 web submission → durable commit and confirmation <1 s under normal load. Inject a pre-commit storage failure: no success confirmation may be emitted; a retry must not create duplicate items. |
| Q6 / I2 | For connected mobile and web clients, p95 database commit → versioned update displayed <=2 s. Change a record through an external writer, test actual push delivery and message ordering, then disconnect/reconnect a client and verify authoritative resynchronization. |
| Q7 / R1, I2 | Run at least 100 paired same-item update attempts with competing bids/edits: zero lost acknowledged writes, zero inconsistent item/borrower/bid combinations, and one ordered outcome per record. The first commit is published before a stale second request is applied; connected-client display freshness remains <=2 s, and conflicts are explicitly rejected or revalidated. |

Photo uploads need a separately approved payload/network budget and are not silently included in a metadata-only one-second test. In Q7, publish means creating the ordered post-commit notification before processing the next conflicting change; it does not mean waiting forever for every disconnected client to acknowledge delivery. If stakeholders require confirmed receipt by all clients first, record that stronger requirement and reevaluate its availability/throughput cost.

# 6. Decisions, evidence and follow-up

| Finding | Follow-up decision / validation |
|---|---|
| SP1 — Search sensitivity | Prototype indexed/paginated server queries; measure database, transport and rendering portions separately. Owner: performance/development lead. |
| RK1 — External account durability | Implement authoritative account persistence with uniqueness and commit-before-success. Owner: backend lead; validate with cross-client and failure-injection tests. |
| RK2 — Identity ownership | Agree an authentication/session protocol and enforce ownership rules on the application server. Owner: product/security lead; preserve the unique-username requirement without treating it as a credential. |
| NR1 — Literal empty fields | Retain fail-fast local validation and duplicate it server-side for remote writes. Owner: mobile/backend leads; measure feedback and separately test malformed values. |
| TP1 — Commit versus response latency | Choose durable confirmation, then optimize the measured path; do not trade truthful success for unacknowledged writes. Owner: product/backend leads. |
| RK3 — Cross-client freshness | Specify commit-triggered events, versioning and reconnect resync. Owner: integration lead; local Observer alone is insufficient. |
| RK4 — Concurrent update integrity | Specify per-item transaction/version semantics and ordered notification. Owner: backend/database lead; verify conflicting bids and item-state changes. |

## Source evidence register

| ID | Inspected evidence in supplied Android archive |
|---|---|
| E1 | SearchActivity.keywordSearch(), lines 88-116: reloads items and performs nested item-word/keyword matching. ItemList.getSearchItems(), lines 90-97: excludes owned and borrowed items. |
| E2 | UserList.loadUsers(), lines 102-117, and saveUsers(), lines 123-141: JSON serialization to app-local files; no shared remote account service is implemented here. |
| E3 | LoginActivity.login(), lines 44-80: known username resolves a user ID and opens MainActivity without an ownership credential; this satisfies identification, not strong authentication. |
| E4 | AddItemActivity.saveItem(), lines 70-97, and validateInput(), lines 123-165: empty required strings set field errors and exit before saving; navigation follows a successful save result. |
| E5 | ItemList.loadItems(), lines 122-137, and saveItems(), lines 140-157: read/write the local item collection, not a remote transactional database. |
| E6 | Observable.notifyObservers(), lines 17-23: iterates local Observer objects and calls update(); no remote event propagation is provided by that mechanism. |
| E7 | ViewItemBidsActivity.acceptBid() and rejectBid(): item, borrower/status and bid changes are coordinated by local controllers/commands; no cross-client database transaction or version protocol is established by this code. |

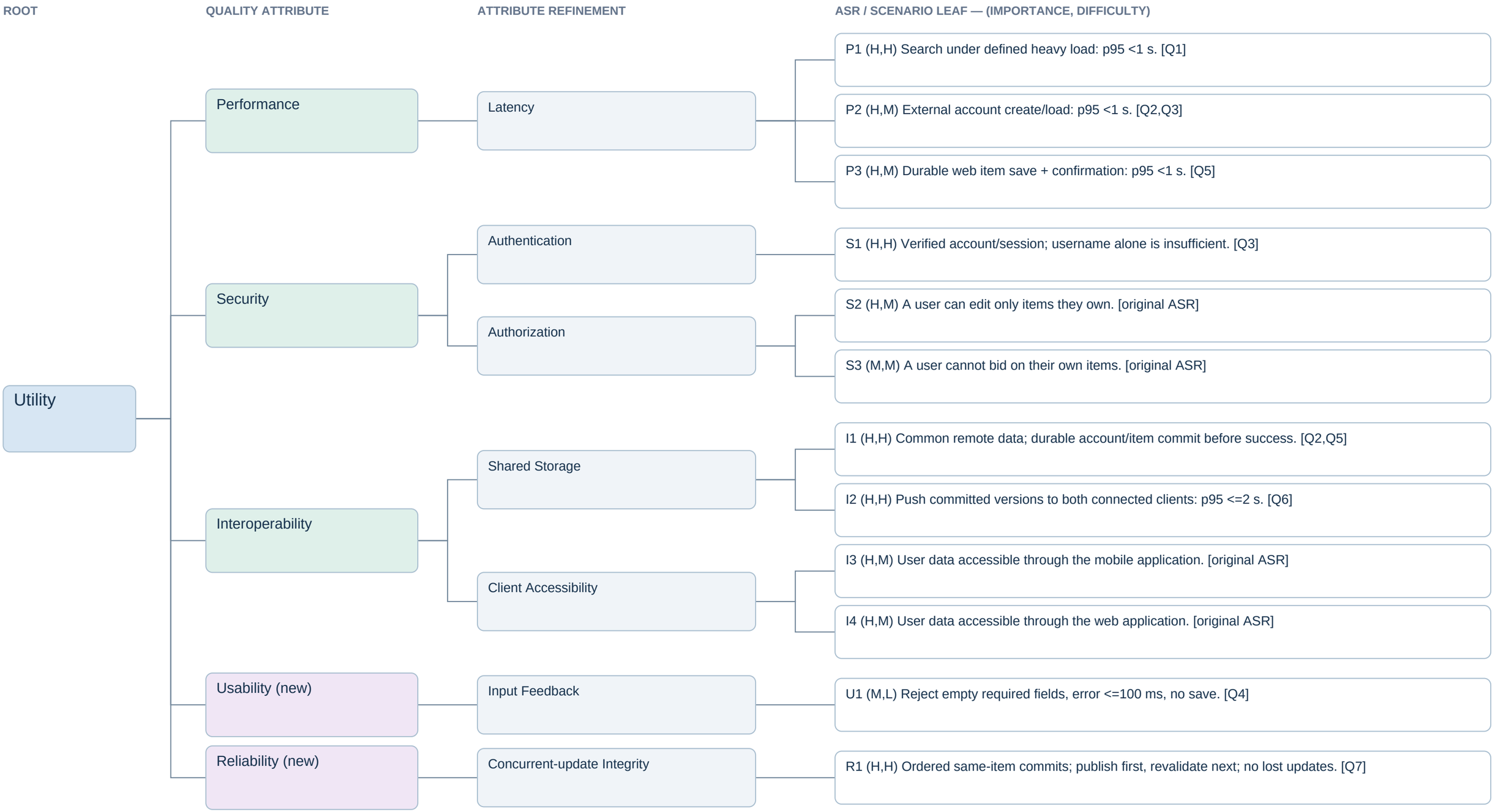
References: supplied Quality Attribute Scenarios.pdf (Q1-Q7); supplied Capstone3_UserStories (2).pdf (US 1-21); supplied SharingApp_c3_local_storage.zip; initial utility-tree image in assignment; earlier component/deployment assignments for the conceptual remote request path. Earlier diagram labels are modeling choices, not proof that remote code exists.

Conclusion: the literal empty-field check is supported, while remote account integrity, identity ownership, propagation and concurrent state changes remain unresolved architectural risks. The utility tree prioritizes these risks alongside measurable latency and usability goals; implementation and stakeholder-reviewed experiments are required before claiming the target architecture satisfies them.

# SharingApp — Updated ATAM Utility Tree

**Fariza Arstanbek**

Utility → Quality Attribute → Attribute Refinement → Architecturally Significant Requirement




**Five changes: C1 latency refinement; C2 stronger authentication; C3 durable shared storage + push; C4 usability; C5 concurrency reliability.**

H/M/L = high/medium/low; pairs = (business importance, estimated architectural difficulty). Targets are proposed, not measured results.

The companion evaluation defines normal/heavy load, measurement boundaries, priorities, all Q1-Q7 findings and source evidence.